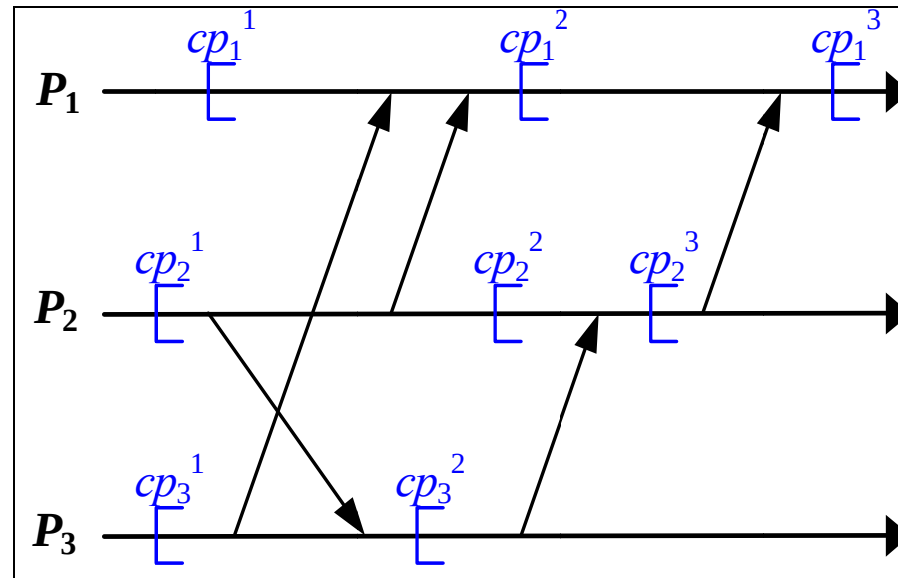




NIEZALEŻNE (ASYNCHRONICZNE) TWORZENIE PUNKTÓW KONTROLNYCH

Sposób wyznaczania CP:

- każdy C_i wyznacza cp_i niezależnie od innych ...
- ... co nie gwarantuje ustalenia CP^*



- trzeba wyznaczyć $RL = \langle cp_1, cp_2, \dots, cp_n \rangle \in GC^*$ (a najlepiej RL^*)



Sposób wyznaczania RL:

- trzeba odszukać CP^* w zbiorze niezależnie wyznaczonych cp_i
 - np. metodą kolejnych wycofań
 - możliwy efekt domino!
-
- albo wycofać się do pewnego CP niekoniecznie spójnego
 - i wyznaczyć RL z precyzją do pojedynczych zdarzeń
 - odtwarzając zdarzenia z rejestru utrzymywanego wraz z punktami kontrolnymi



Punkty odtwarzania

- punkty kontrolne (*checkpoints*)

W najprostszym przypadku punkt odtwarzania = punkt kontrolny.

- rejestrowanie zdarzeń i odtwarzanie z rejestrów (*logging and replaying*)

Jednakże punkt odtwarzania można osiągnąć wycofując proces do poprzedzającego punktu odtwarzania i przesuwając wykonanie przetwarzania poprzez odtworzenie zdarzeń (niedeterministycznych) zapamiętanych w rejestrze, aż do osiągnięcia odpowiedniego stanu (który stanowić będzie punkt odtwarzania).

Linia odtwarzania

Globalny punkt odtwarzania = wektor lokalnych punktów odtwarzania

Linia odtwarzania = **spójny** globalny punkt odtwarzania

$$\mathbf{RL} = \langle rp_1, rp_2, \dots, rp_n \rangle$$



Algorytm Juanga-Venkatesana [1]

Założenia:

- kanały niezawodne z uporządkowaniem komunikatów
- nieskończona pojemność kanałów

Idea:

- wyszukujemy rp_i z dokładnością do pojedynczych zdarzeń komunikacyjnych
- detekując osierocone wiadomości przez porównywanie ilość odebranych wiadomości z ilością wiadomości wysłanych

Wykorzystywane struktury:

- $SENT_{i \rightarrow j}(rp_i)$ = ilość wiadomości wysłanych przez P_i do P_j do punktu rp_i
 - $RCVD_{i \leftarrow j}(rp_i)$ = ilość wiadomości odebranych przez P_i od P_j do punktu rp_i
 - przy każdym zdarzeniu wysłania lub odbierania wiadomości proces zapamiętuje tę wiadomość w logu
- wartości tych struktur są zapisywane w pamięci trwałej razem z cp_i

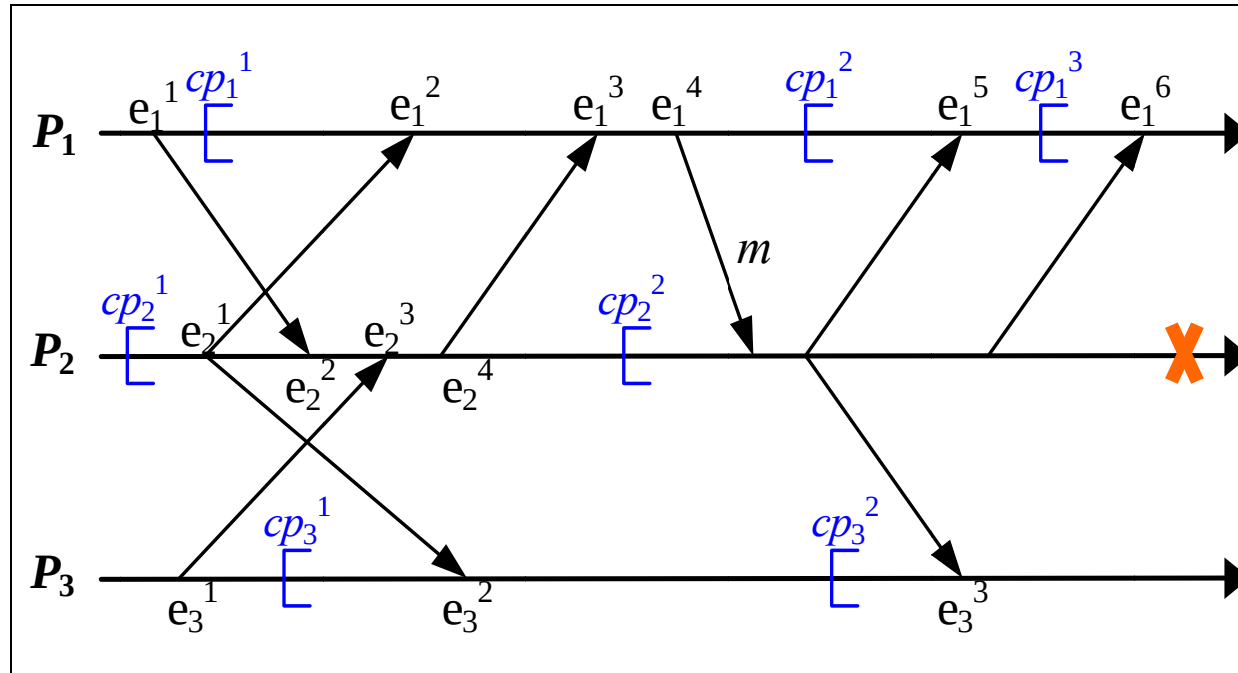


Przebieg algorytmu odtwarzania:

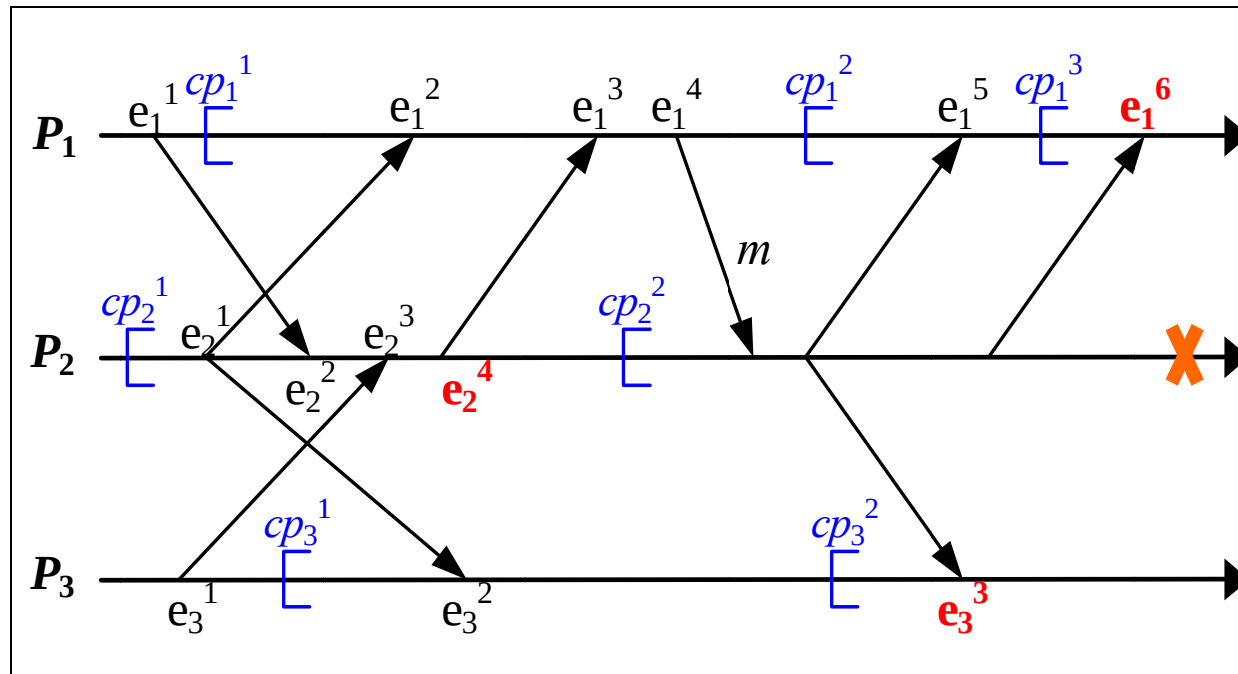
- Kontroler C_f restartowanego po awarii procesu P_f informuje o swym restarcie pozostałe procesy rozsyłając broadcast (złożoność komunikacyjna $O(|E|)$, gdzie $|E|$ jest całkowitą liczbą łączy komunikacyjnych).
- Wykonanie algorytmu przez C_i rozpoczyna się gdy P_i jest restartowany lub gdy C_i dowiaduje się o restarcie innego P_f . Zatem algorytm jest inicjowany jednocześnie przez wszystkie kontrolery procesów. Inicjacja polega na wyznaczeniu początkowej wartości rp_i^1 .
- Algorytm wykonuje się w N iteracjach („rundach”), gdzie N jest liczbą procesów. W k -tej iteracji ($k > 1$) kontroler C_i wykorzystując rp_i^{k-1} wyznaczony w iteracji $k-1$, oblicza wartości $SENT_{i \rightarrow j}(rp_i^{k-1})$ dla każdego sąsiedniego procesu P_j , wysyłając ją następnie w komunikacie *ROLLBACK*. Przetwarzając komunikaty *ROLLBACK* odebrane od sąsiadów, odnajduje taki rp_i^k aby dla każdego sąsiedniego procesu P_j $RCVD_{i \leftarrow j}(rp_i^k) \leq SENT_{j \rightarrow i}(rp_i^{k-1})^*$
- Po zakończeniu każdej rundy, co najmniej jeden proces wycofa się do swojego ostatecznego lokalnego punktu odtwarzania, za wyjątkiem przypadków, gdy aktualnie wyznaczone punkty odtwarzania już stanowią spójną linię odtwarzania.

* precyzyjniej: $RCVD_{i \leftarrow j}(rp_i^k) = SENT_{j \rightarrow i}(rp_i^{k-1})$, ew. za wyjątkiem pierwszej iteracji w procesie restartowanym

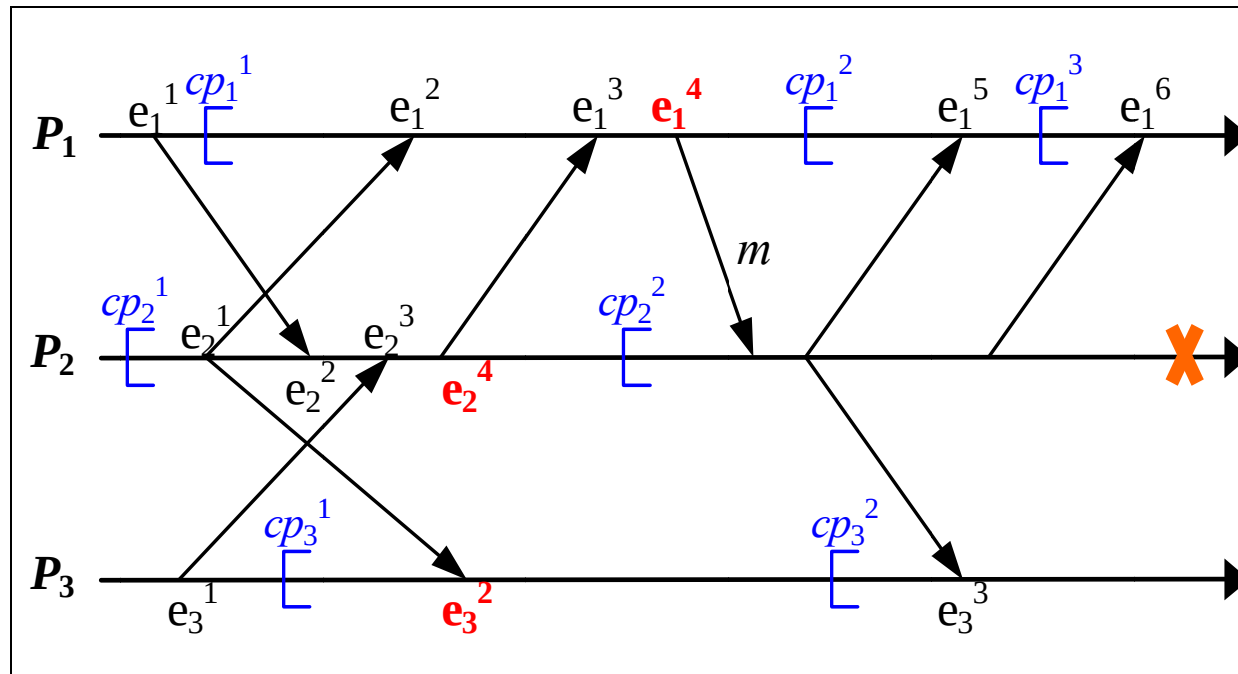
Przykład:



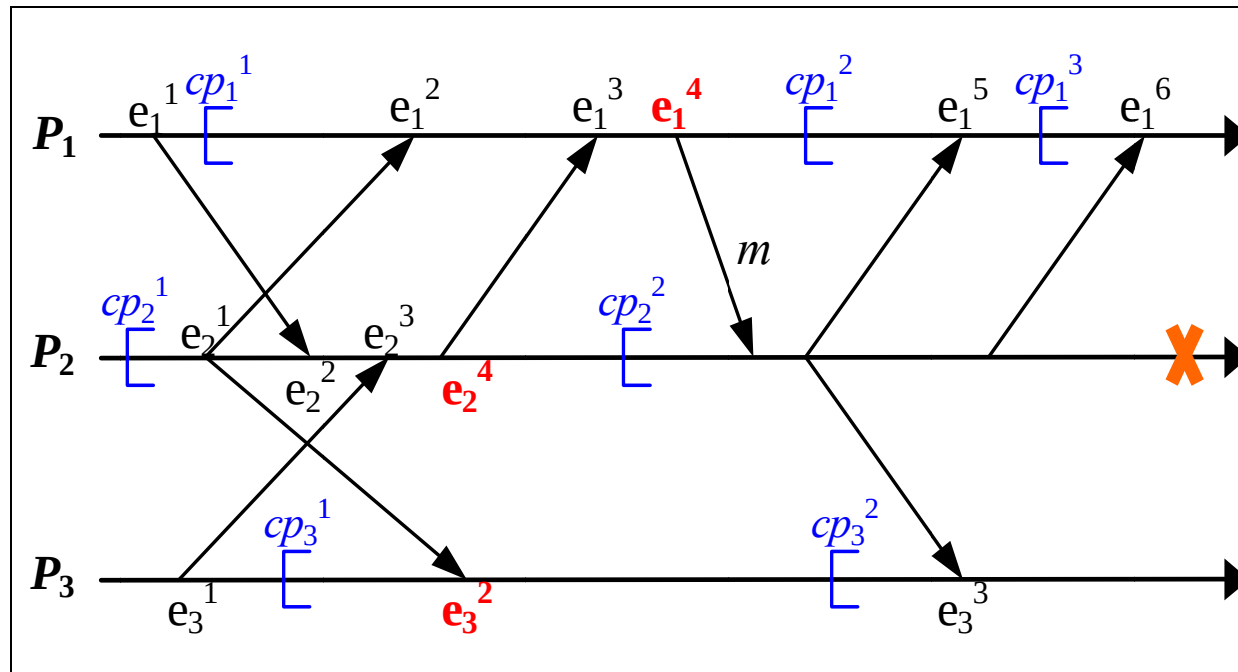
1. Proces P_2 po awarii jest restartowany z punktu cp_2^2 , który odpowiada stanowi po zdarzeniu e_2^4 . Wysłany broadcast inicjalizuje algorytm również na P_1 i P_3 . Mamy $rp_1 = e_1^6$, $rp_2 = e_2^4$ oraz $rp_3 = e_3^3$ (e_1^6 i e_3^3 są wirtualnymi punktami odtwarzania).



2. C_2 wysłała $ROLLBACK(C_2, C_1, 2)$ do C_1 i $ROLLBACK(C_2, C_3, 1)$ do C_3 .
3. C_1 wysłała $ROLLBACK(C_1, C_2, 2)$ do C_2 i $ROLLBACK(C_1, C_3, 0)$ do C_3 .
4. C_3 wysłała $ROLLBACK(C_3, C_1, 0)$ do C_1 i $ROLLBACK(C_3, C_2, 1)$ do C_2 .



5. Ponieważ $RCVD_{1 \leftarrow 2}(e_1^6) = 4 > 2$, więc C_1 wyznacza nowy $rp_1 = e_1^4$. Teraz $RCVD_{1 \leftarrow 2}(e_1^4) = 2$.
6. Ponieważ $RCVD_{3 \leftarrow 2}(e_3^3) = 2 > 1$, więc C_3 wyznacza nowy $rp_3 = e_3^2$. Teraz $RCVD_{3 \leftarrow 2}(e_3^2) = 1$.
7. Ponieważ $RCVD_{2 \leftarrow 3}(e_2^4) = 1 = SENT_{3 \rightarrow 2}(rp_3)$ oraz $RCVD_{2 \leftarrow 1}(e_2^4) = 1 < 2$, więc P_2 nie potrzebuje wycofywać się głębiej. Zatem, w drugiej iteracji C_2 wysyła ponownie $ROLLBACK(C_2, C_1, 2)$ do C_1 i $ROLLBACK(C_2, C_3, 1)$ do C_3 .

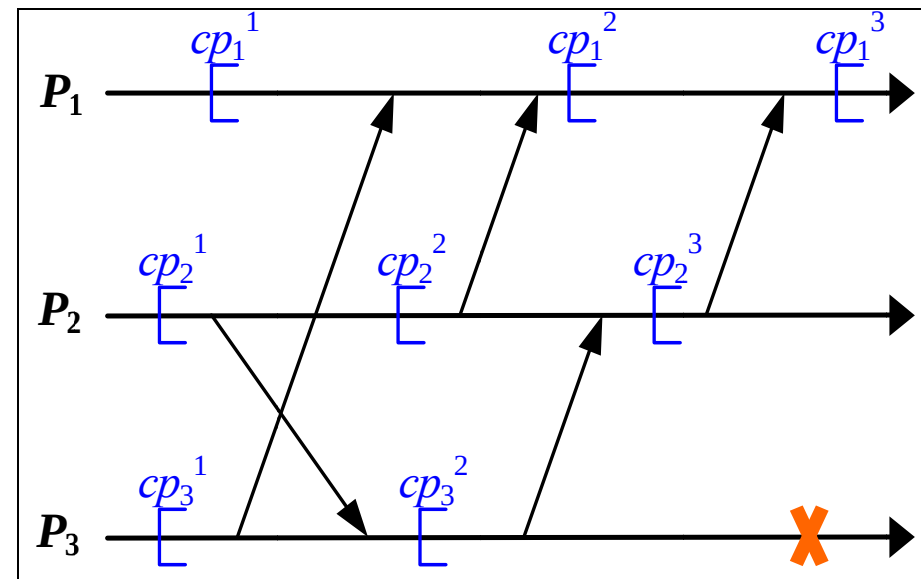


8. C_1 wysła $ROLLBACK(C_1, C_2, 2)$ do C_2 i $ROLLBACK(C_1, C_3, 0)$ do C_3 .
9. C_3 wysła $ROLLBACK(C_3, C_1, 0)$ do C_1 i $ROLLBACK(C_3, C_2, 1)$ do C_2 .
10. Trzecia runda przebiega identycznie. Zatem ostateczna linia odtwarzania (e_1^4, e_2^4, e_3^2) została wyznaczona w pierwszej rundzie i po niej nic się już nie zmienia.
11. Wiadomość m zostanie odtworzona z logu procesu P_1 .



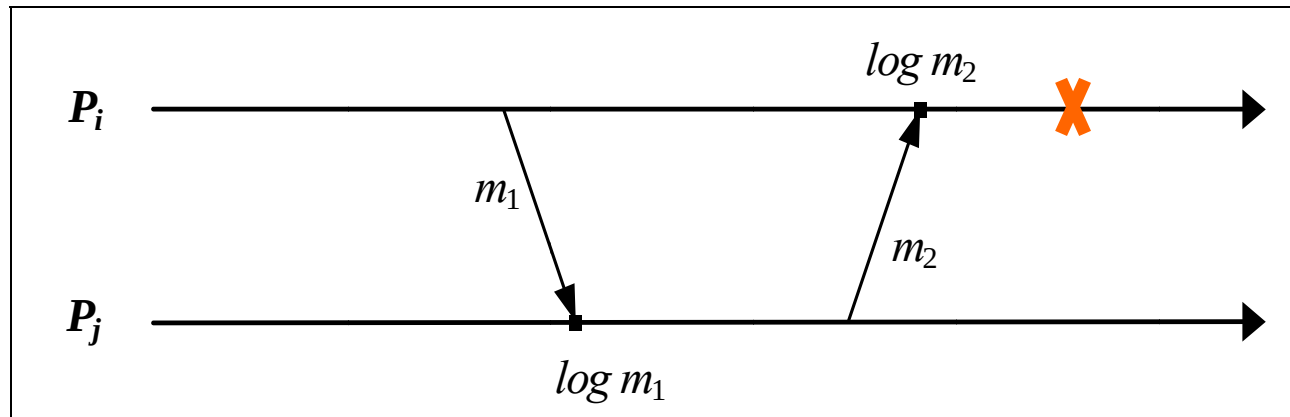
Rejestrowanie wiadomości

- pozwala ograniczyć rollback
- i uniknąć efektu domino
- po wycofaniu do cp_i odtwarzając zarejestrowane wcześniej wiadomości
- aż do osiągnięcia stanu rp_i , w którym żadna osierocona wiadomość nie została odebrana



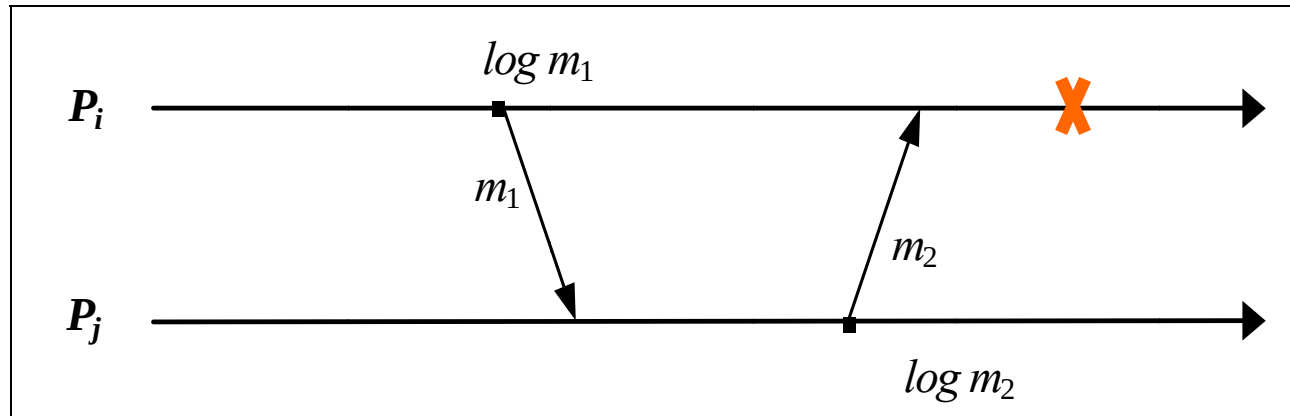


Rejestrowanie po stronie odbiorcy:

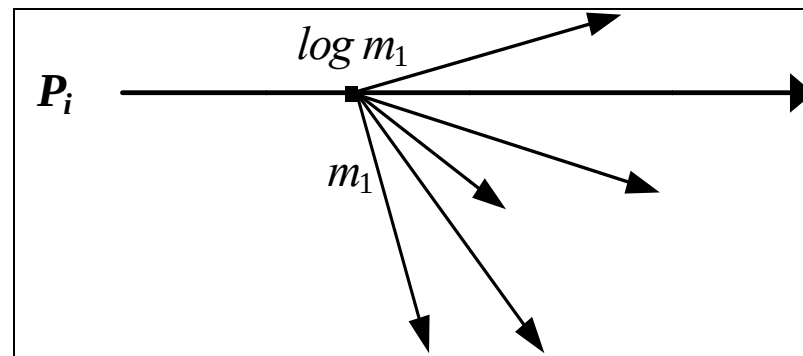


- *receive* jest operacją niedeterministyczną
- przy odtwarzaniu czekamy na wiadomość właśnie po stronie odbiorcy
– odtworzenie wiadomości z logu (*replay*) przyspiesza ponowne wykonanie przetwarzania (*re-execution*)
- retransmisja zagubionej m_2 może nie być konieczna

Rejestrowanie po stronie nadawcy:

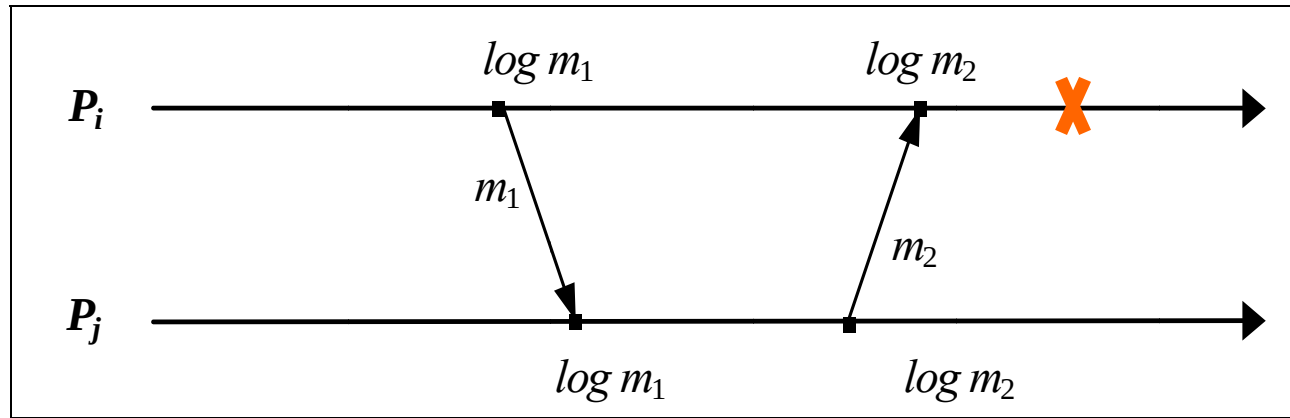


- usprawnia retransmisję (skądinąd potrzebną)
- umożliwia przetwarzanie niedeterministyczne
- broadcast





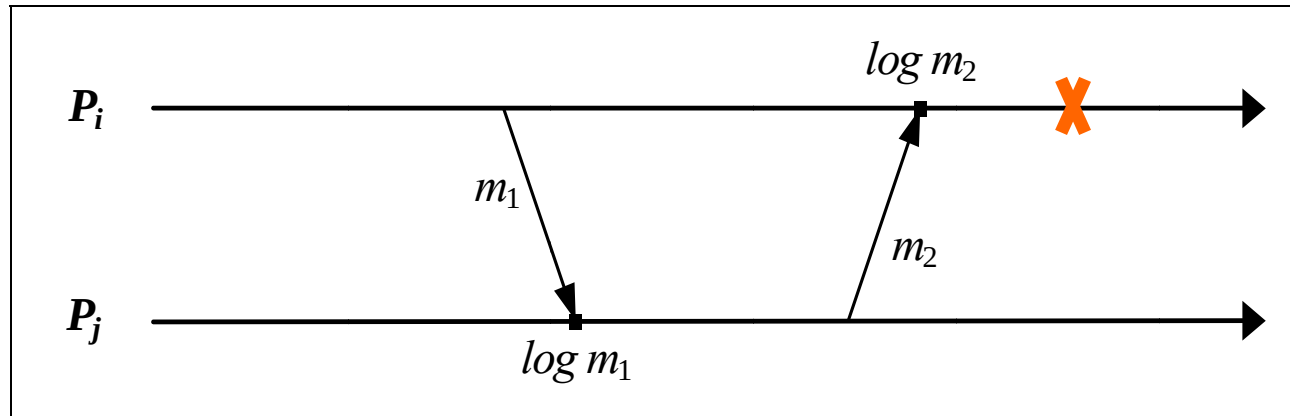
Rejestrowanie jednocześnie po stronie nadawcy i odbiorcy:



- nadmiar ?



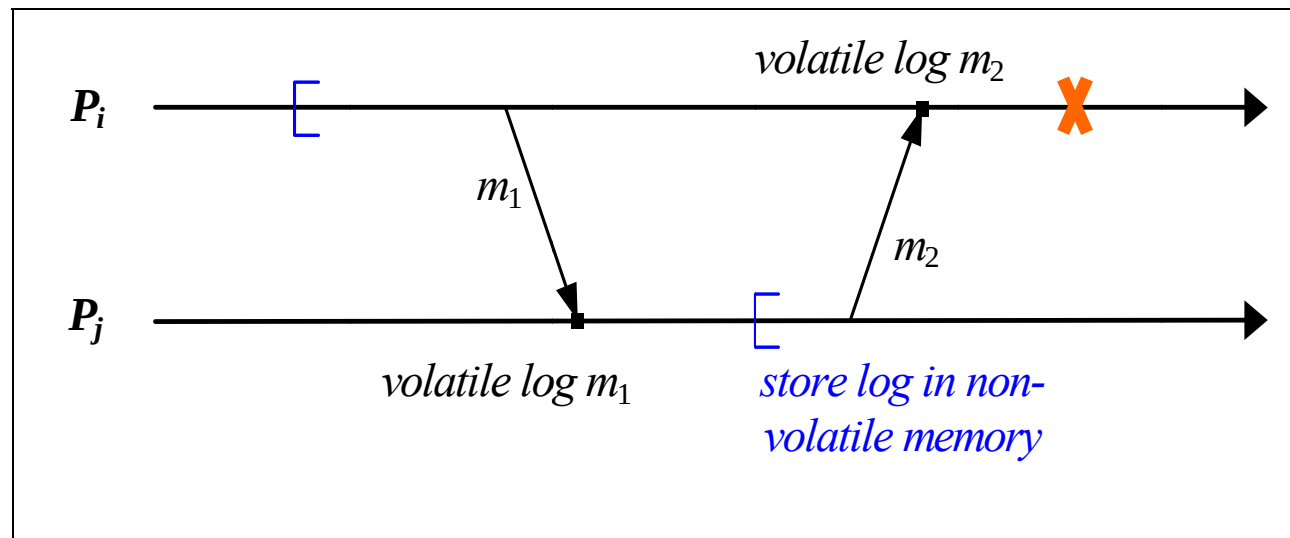
Rejestrowanie pesymistyczne:



- zapis logu do pamięci trwałej atomowo z operacją *send* / *receive*
- atomowo w praktyce $\rightarrow C_i: \log$ po *receive*, ale przed *deliver*
- duży narzut czasowy (blokowanie przetwarzania aplikacyjnego)



Rejestrowanie optymistyczne:

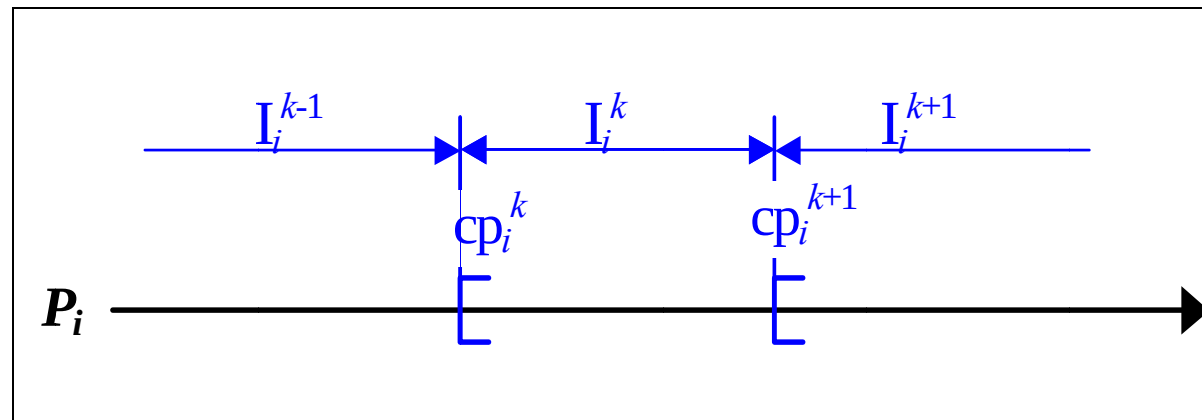


- wraz z operacją *send / receive* zapis do rejestru w pamięci ulotnej
- rejestr przepisany do pamięci trwałej przy późniejszej okazji
- w przypadku awarii, wraz z ulotnym rejestrem utracone zostają niektóre wiadomości



Dependency of checkpoints

Checkpoint interval:





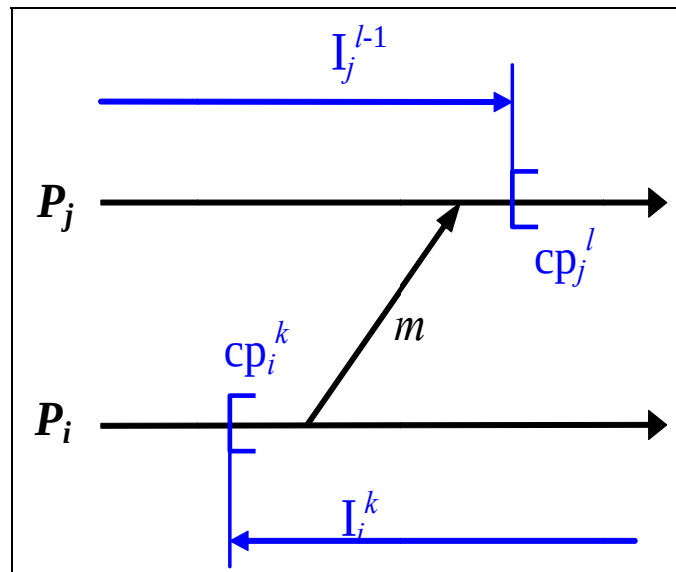
Z-dependency (zigzag-dependency) relation:

cp_j^l is *directly z-dependent* on $cp_i^k \Leftrightarrow$

$$i=j \wedge l=k+1$$

or

$$i \neq j \wedge \exists m: (\text{send}(P_i, \{P_j\}, m) \in I_i^k \wedge \text{recv}(P_j, P_i, m) \in I_j^{l-1})$$



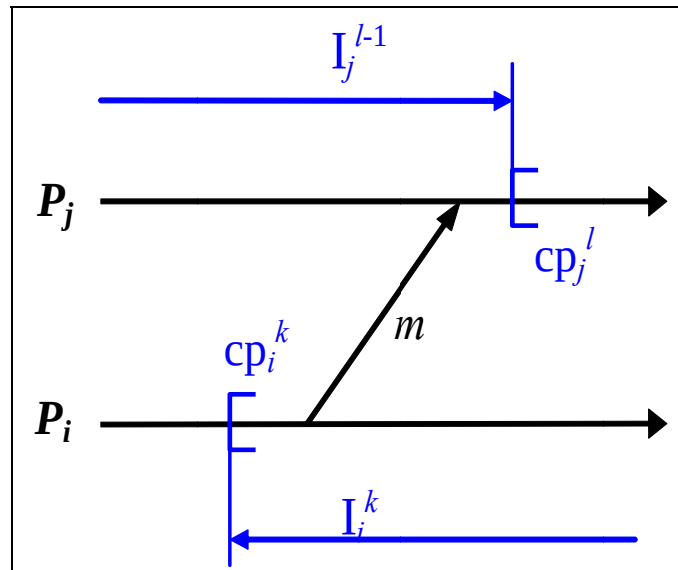
Z-dependency of checkpoints is transitive closure on *direct z-dependency* relation.



Zigzag-dependency vs. consistency:

Theorem 1:

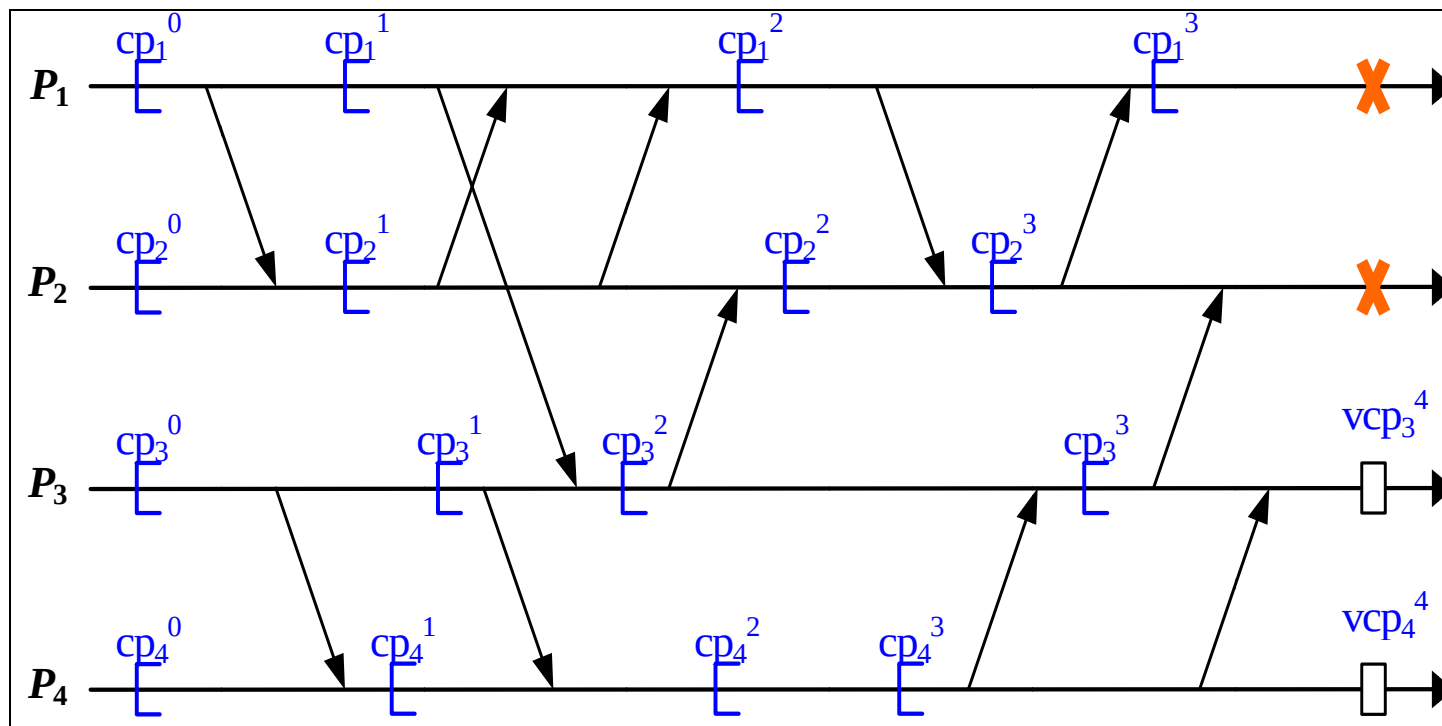
Z-dependency between two checkpoints makes them an inconsistent checkpoint pair.

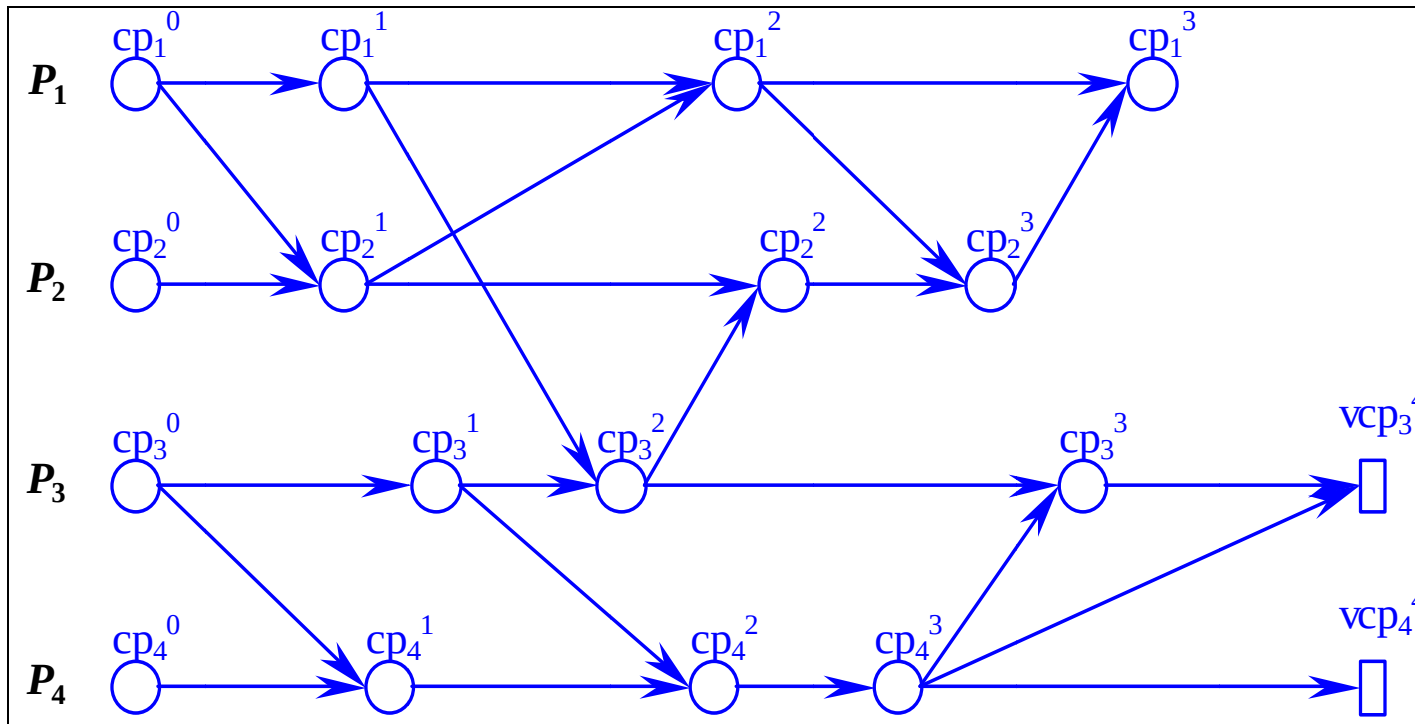




Checkpoint (z-dependency) graph:

- nodes represent checkpoints (checkpoint intervals)
- edges represent direct z-dependency relation of checkpoints





- **extended** checkpoint graph includes virtual checkpoints



Wang-Fuchs Algorithm [2]

System model:

- fail-recovery model of process failures
- reliable reordering channels
- no network partitioning
- deterministic processing (can be relaxed)
- all activities can be undone (output commit, if necessary)

General idea:

- use of checkpoint z-dependency graph
- z-dependency tracking – all messages tagged with additional control information
- optimistic message logging (receiver logs message content)
- some control data saved in stable storage together with checkpoints and message logs



Structures:

- $ckpt_num_i$ – checkpoint interval number of P_i
- $send_num_i$ – seq. number of application messages sent by P_i
- $send_log_i$ – contains elements $(send_num_i, j)$ where P_j is the destination of the message m sent by P_i with seq. number $send_num_i$ (also with $ckpt_num_i$)
- rec_log_i – contains elements $(m, send_num_j, ckpt_num_j, j)$ describing received message m



Checkpointing:

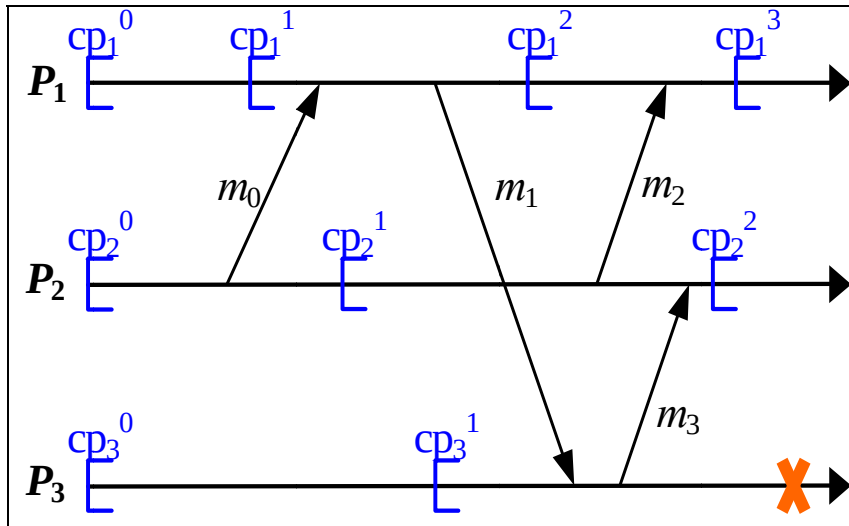
When controller C_i decides spontaneously and independently of other controllers to take another checkpoint of P_i , it stores current process state along with $send_log_i$, rec_log_i and $ckpt_num_i$. Then clears volatile logs $send_log_i$ and rec_log_i .

Recovery:

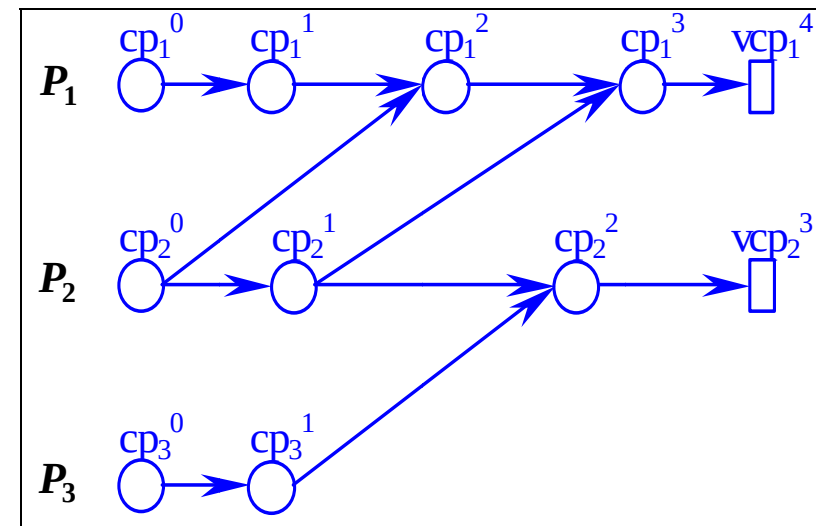
On recovery of a failed process P_f a global checkpoint graph of whole computation must be constructed. C_f requests other controllers to send back the z-dependency information. Requested controller C_i takes a virtual checkpoint and replies with current values of $send_log_i$, rec_log_i and $ckpt_num_i$. When C_f receives all replies, it constructs its extended checkpoint graph.



(space-time diagram)



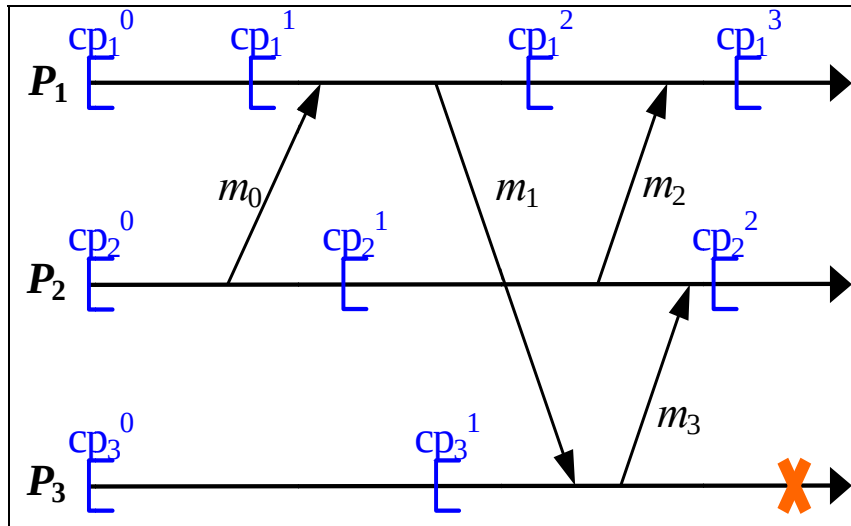
(extended checkpoint graph)



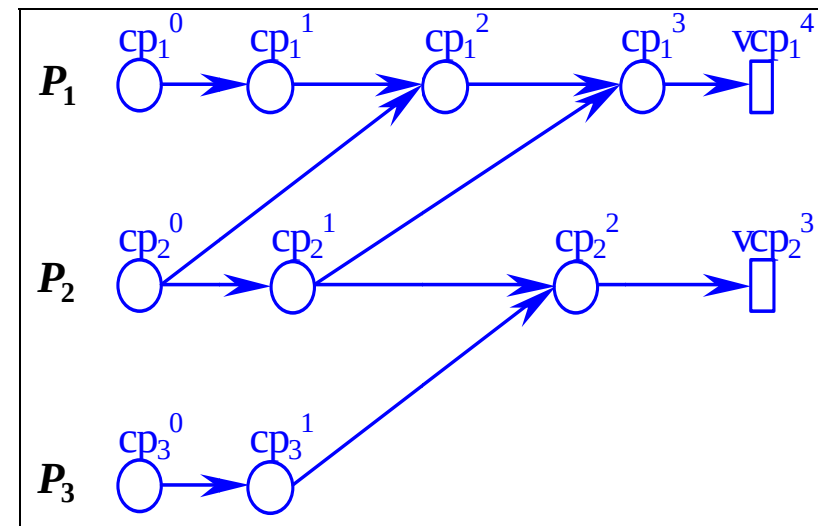
- pair of checkpoints (cp_3^1, cp_2^2) is not consistent due to orphan message m_3
(existence of an orphan message is depicted in checkpoint graph by edges linking checkpoints on different processes)
- pair (cp_2^1, cp_1^3) is also inconsistent
- pairs (cp_2^1, cp_1^2) and (cp_3^1, cp_2^1) are consistent



(space-time diagram)



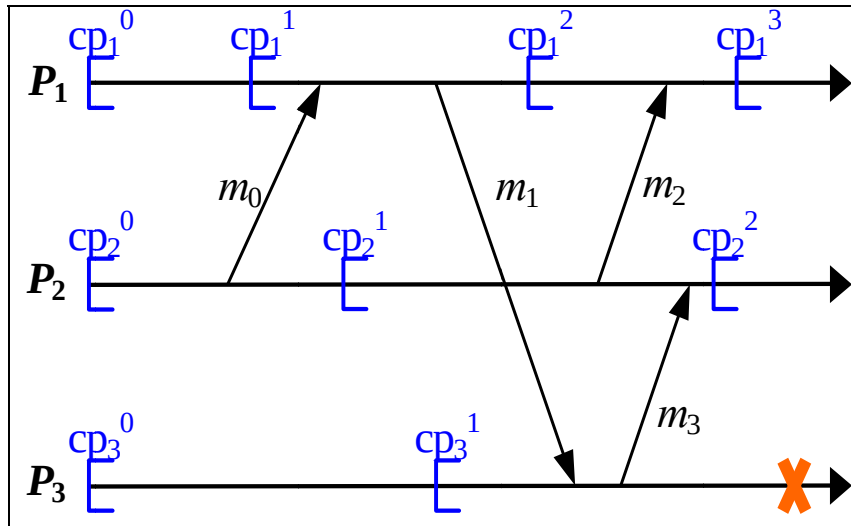
(extended checkpoint graph)



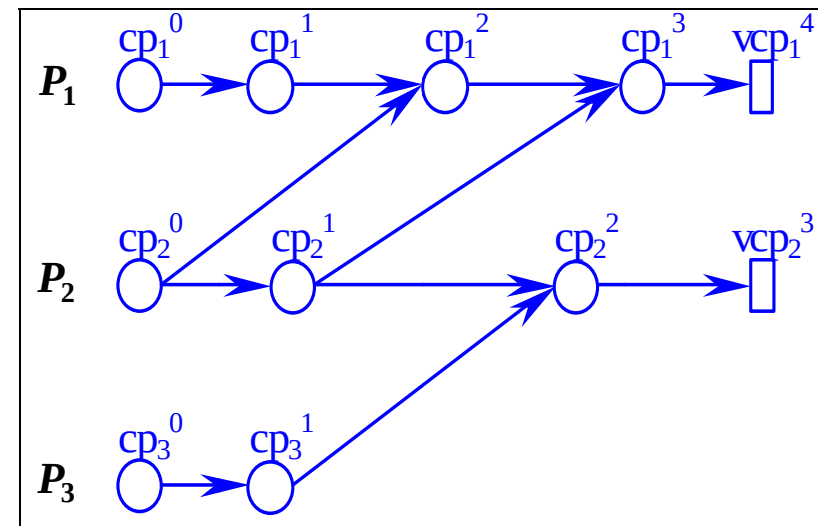
○ what about (cp_3^1, cp_1^2) ?



(space-time diagram)

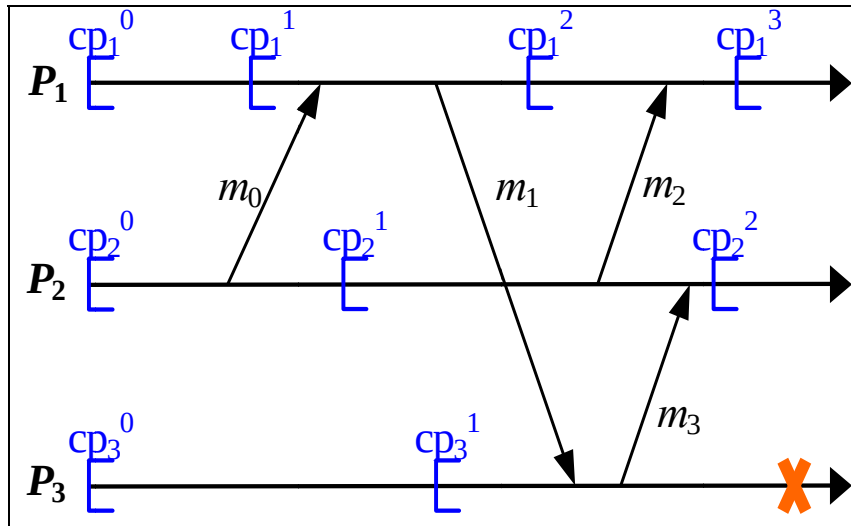


(extended checkpoint graph)

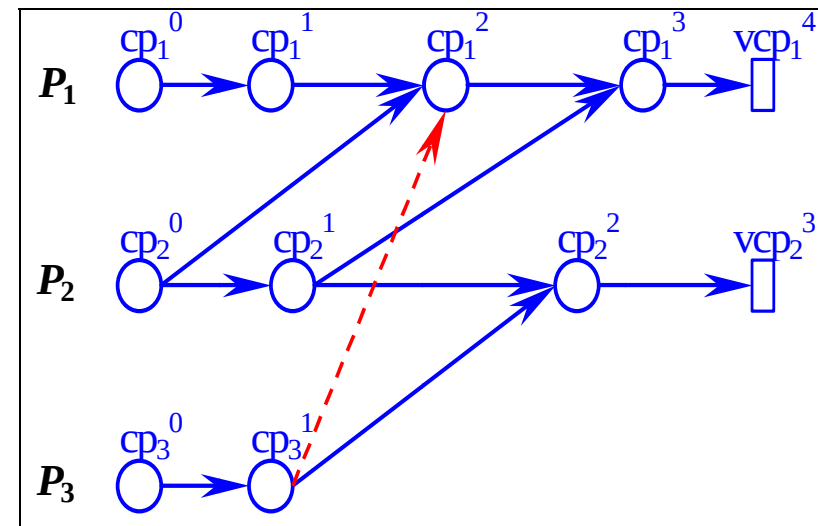


- in result of P_3 's failure the log of receiving m_1 in P_3 has been lost
- restarting P_1 from cp_1^2 will not reproduce m_1 (nor any substitute m'_1)

(space-time diagram)



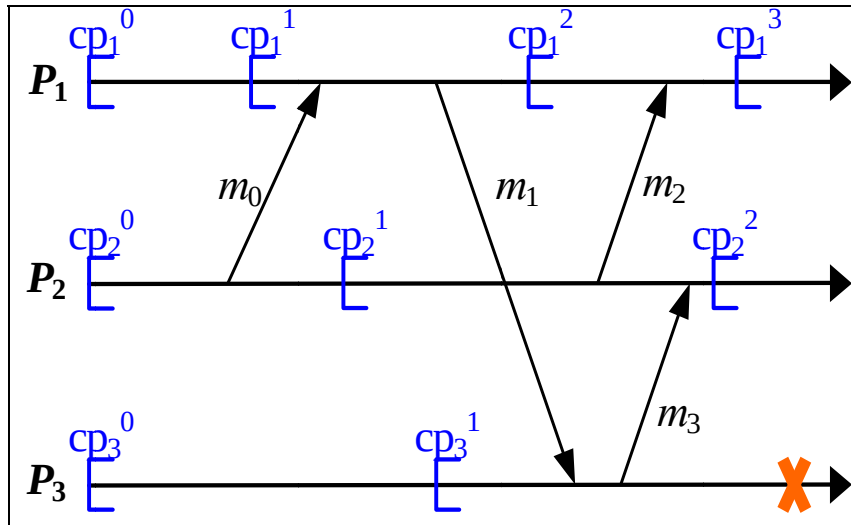
(extended checkpoint graph)



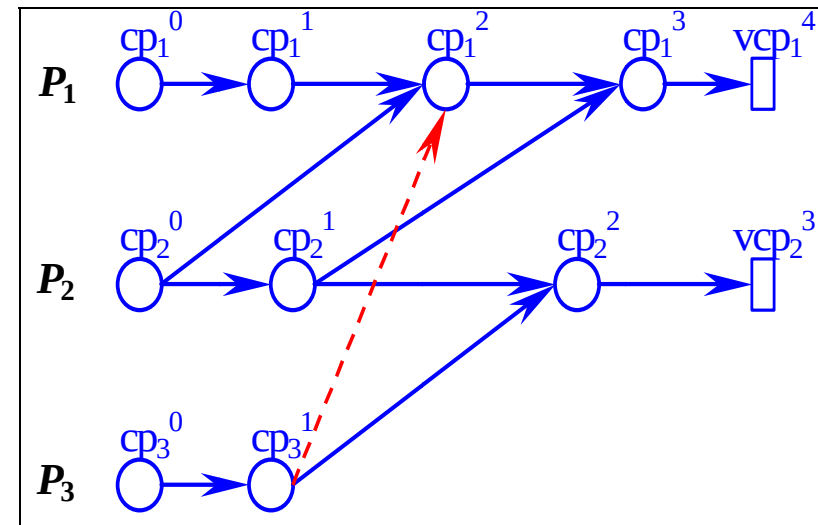
- **rollback edges** represent not-yet-logged messages
- hence P_1 must be rolled back to cp_1^1



(space-time diagram)

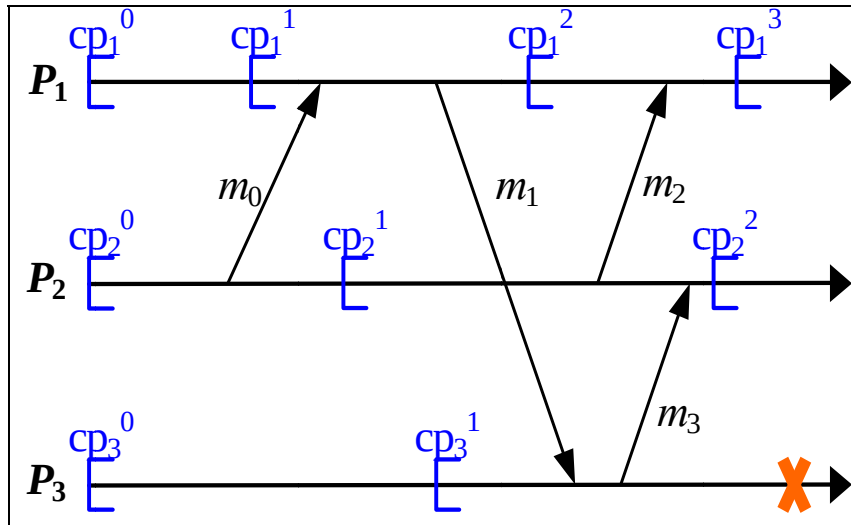


(extended checkpoint graph)

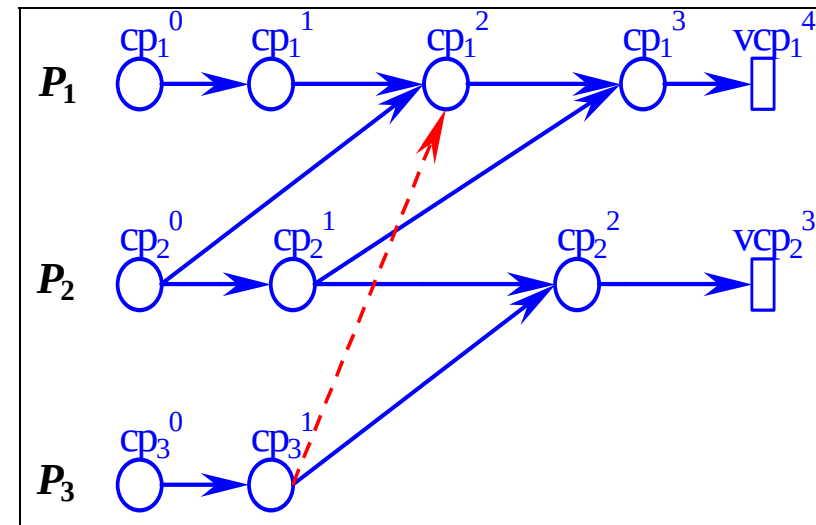


- what about m_0 and rollback edge $cp_1^1 \rightarrow cp_2^1$?

(space-time diagram)



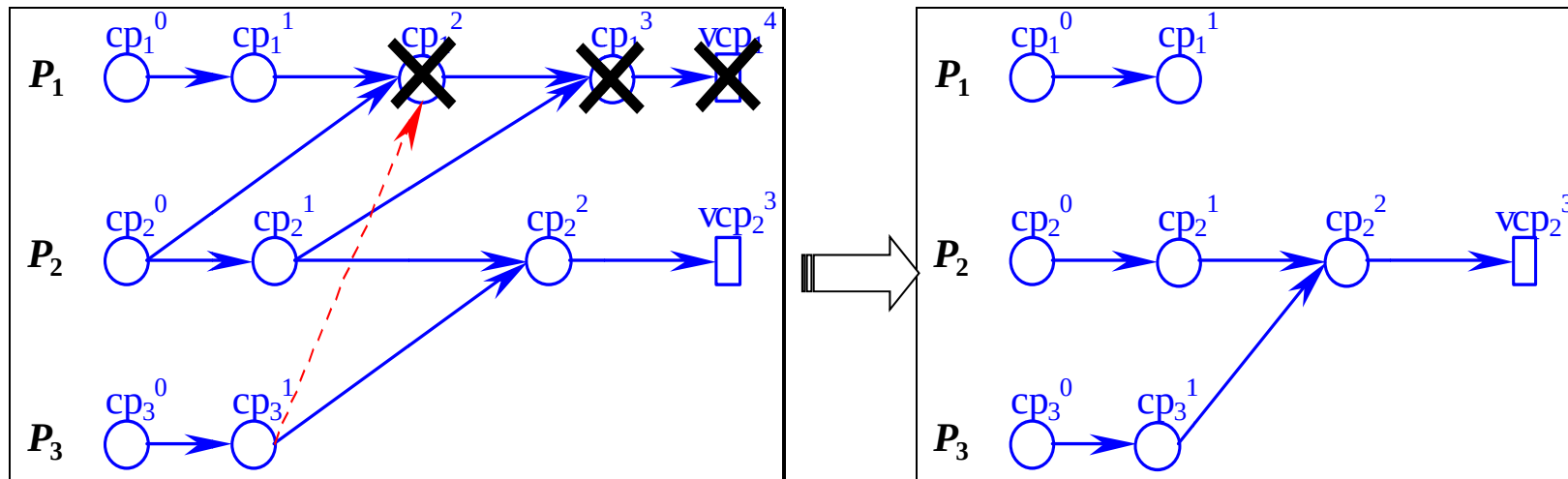
(extended checkpoint graph)



- since m_0 is available in message log rec_log_1 at cp_1^2 and can be replied, there is no rollback edge $cp_1^1 \rightarrow cp_2^1$

Reduced checkpoint graph:

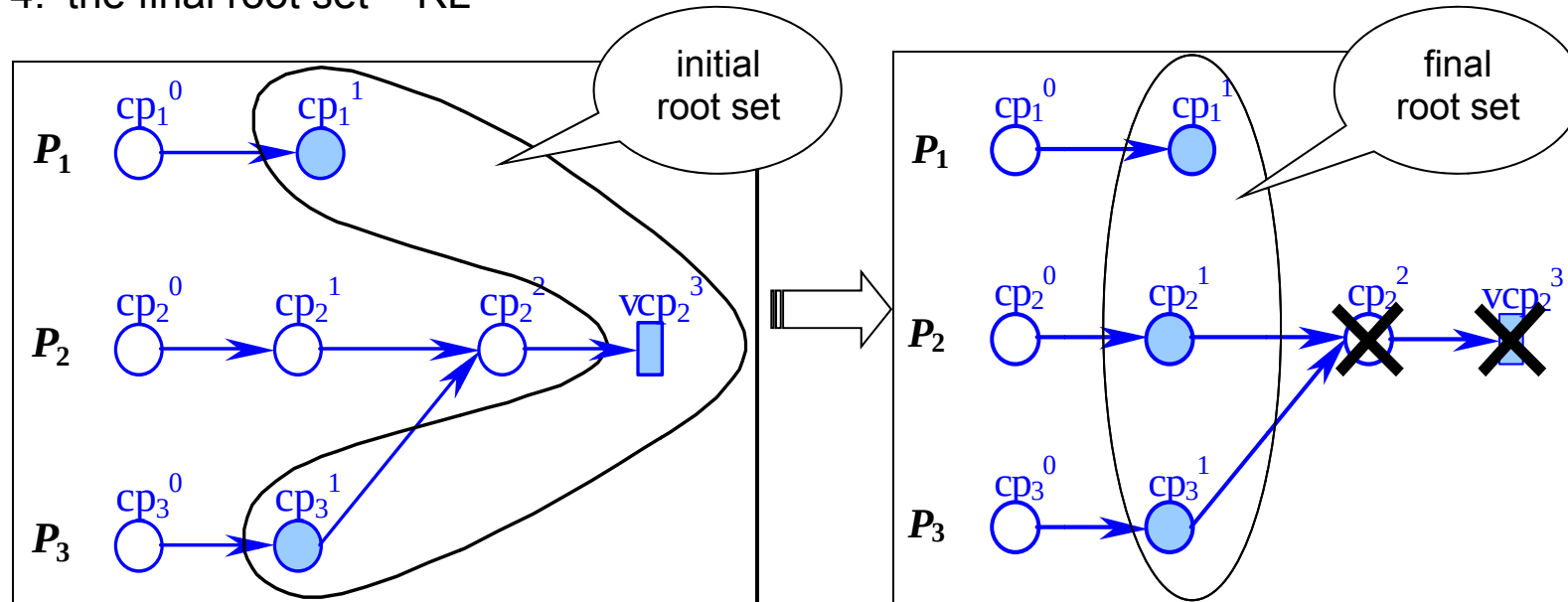
- a node with incoming rollback edge represents a potential inconsistency of checkpoints (until the message is logged)
- therefore we exclude any node with incoming rollback edge as well as all nodes reachable from them:



(remember: the rollback edge is specific to the Wang-Fuchs algorithm!)

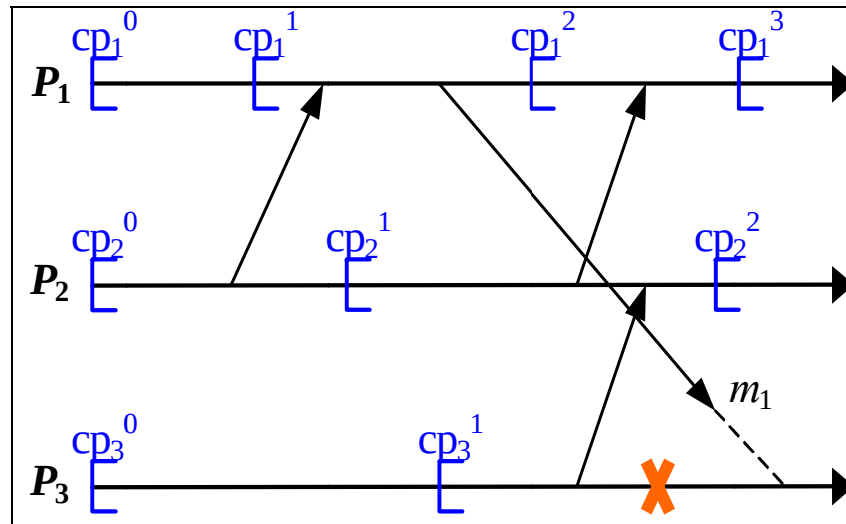
Recovery line search algorithm:

1. include the last checkpoint of each process in a **root set**
2. mark all checkpoints reachable from any checkpoint in the root set
3. while (at least one checkpoint in the root set is marked)
 - {
 - replace each marked checkpoint in the root set by the last unmarked checkpoint of the same process
 - mark all checkpoints reachable from any checkpoint in the root set
 - }
4. the final root set = RL



**Recovery:**

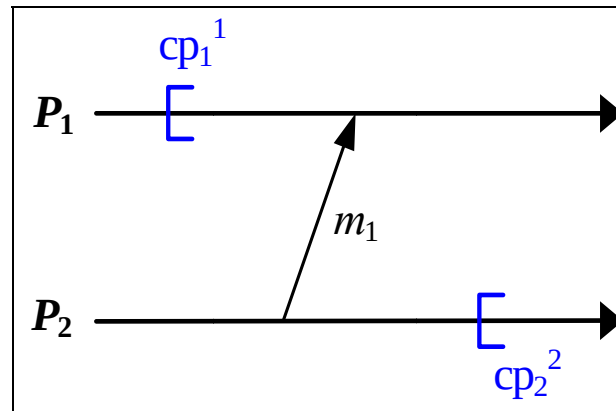
- during re-execution, incoming messages can be replied from rec_log_i until it is exhausted
- outgoing messages are sent again (yet beware of duplicates!)





Reducing message logging overhead

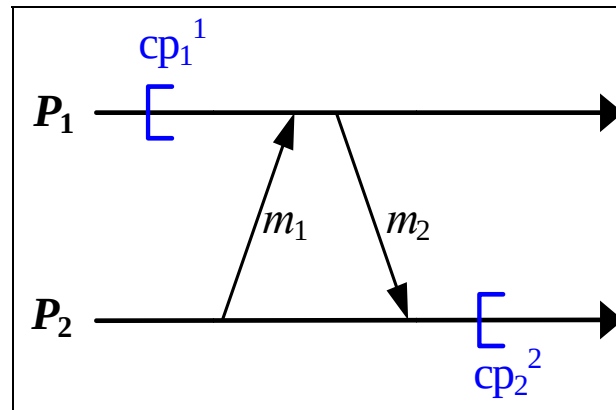
- channel state:



- with respect to (cp_1^1, cp_2^1) m_1 is *in-channel* (*in-transit*, *in-state message*)
- it belongs to channel state corresponding to this pair of checkpoints
- will be lost if not logged



Observation: a message that will never become part of any channel state (*non-state message*) needs not to be logged



- with respect to (cp_1^1, cp_2^2) m_2 is orphan
- due to m_2 (cp_1^1, cp_2^2) is inconsistent
- m_1 is non-state – its availability in logs does not influence consistency
- since regardless of logging m_1 or not the checkpoint pair is not consistent, there is no reason to log m_1 at all



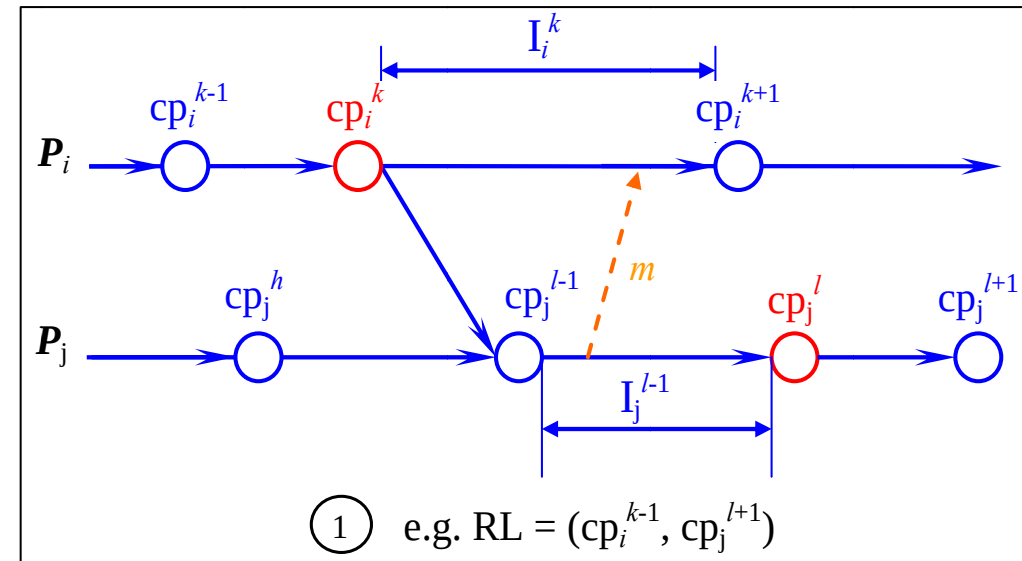
Theorem 2:

If there exists a path from cp_i^k to cp_j^l on the checkpoint graph, then all messages sent from I_j^{l-1} and received at I_i^k are non-state messages.

Proof:

- 1 Choose any m belonging to such a channel state – this channel state does not belong to any RL:

Suppose the channel state corresponding to a recovery line RL contains such message m . By definition, RL must contain $cp_i^{x \leq k}$ and $cp_j^{y \geq l}$, hence there is a path from cp_i^x to cp_i^k and from cp_j^l to cp_j^y . If there also exists a path from cp_i^k to cp_j^l , we have the path from cp_i^x to cp_j^y , what implies these two checkpoints are inconsistent, contradicting the fact that they are on the same recovery line RL.

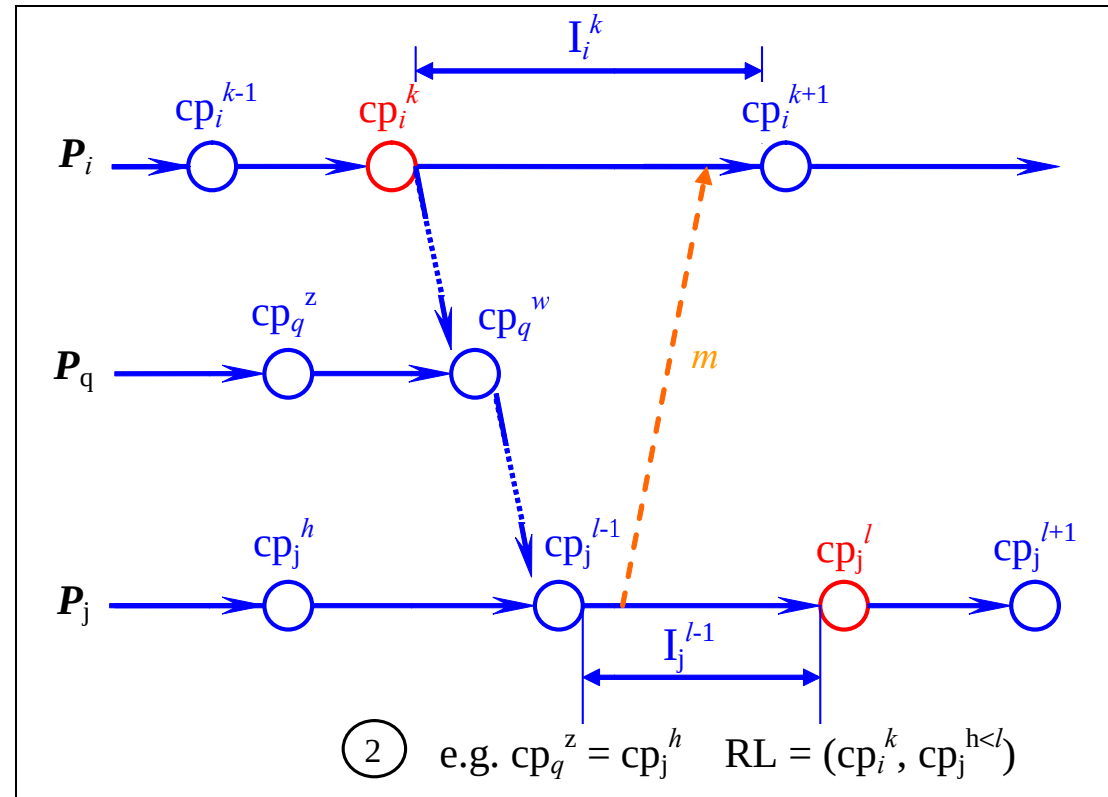




- 2 Choose any RL – no such m can belong to the channel state in this RL:

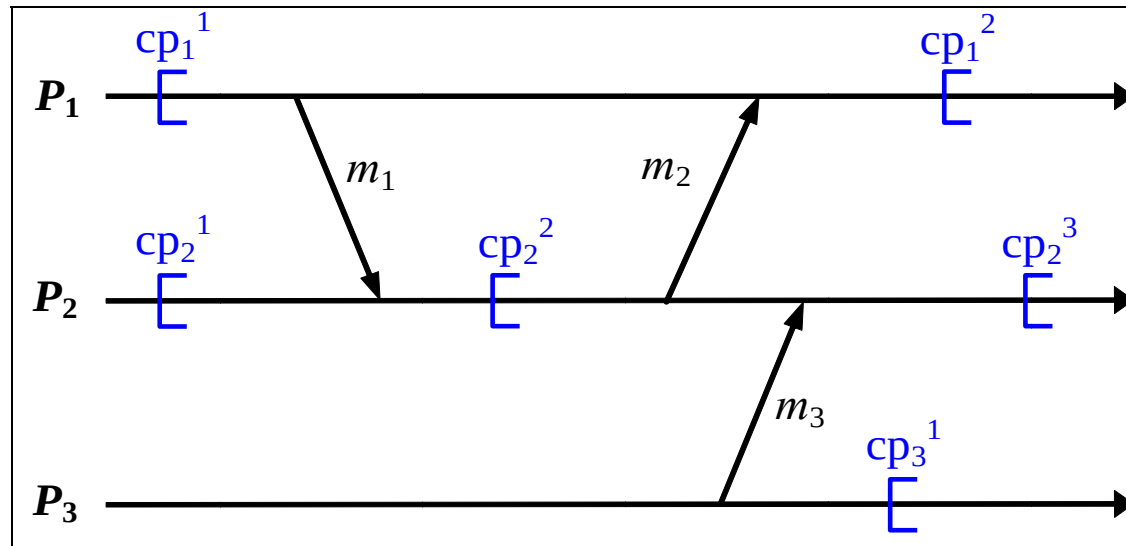
The path from cp_i^k to cp_j^l can be broken only if for any cp_q^w on that path, P_q rolls back to cp_q^z : $z < w$. Then, P_j must roll back $cp_j^{h < l}$ to due to the z-dependency path. All the messages sent from I_j^{l-1} are then invalidated and can never become part of any channel state. Therefore, all the messages satisfying the statement of the theorem are non-state messages.

□





Example



- m_1 is non-state – look at (cp_1^2, cp_2^1)
- m_2 is non-state – look at (cp_1^1, cp_2^3)
- what about m_3 ?
 - actually pair (cp_2^2, cp_3^1) will not constitute a RL (neither with cp_1^1 , nor cp_1^2)
 - but it is unknown to P_3 on reception of m_3
 - clearly by def. m_3 is not non-state – pair (cp_2^2, cp_3^1) is consistent



GARBAGE COLLECTION

Goal:

- to reclaim checkpoints not useful any longer (discardable)

Discardable checkpoints:

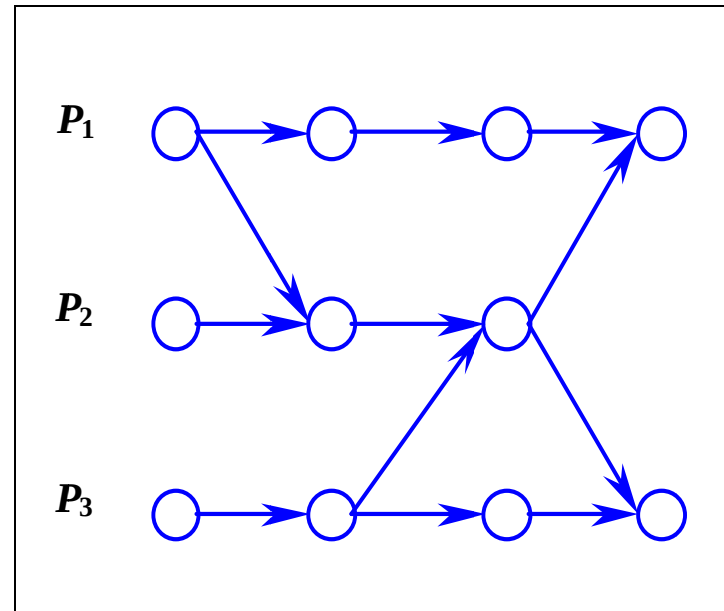
Def. *Discardable* checkpoint is a checkpoint that will never belong to any future recovery line.

Obsolete checkpoints:

- preceding the worst-case recovery line LR
- obsolete checkpoints are clearly discardable
- but some non-obsolete checkpoints may be also discardable

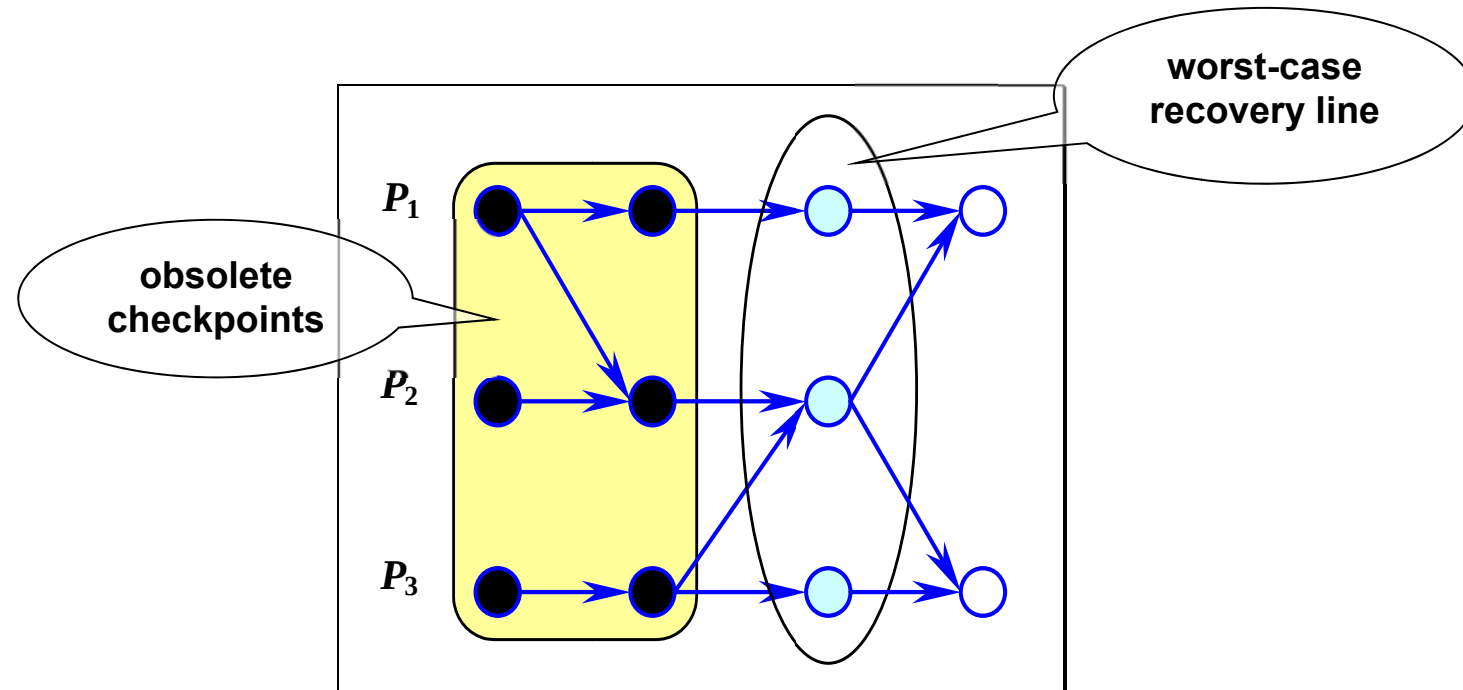


Obsolete checkpoints:



- where is the recovery line found by Wang-Fuchs in the worst case?

Obsolete checkpoints:

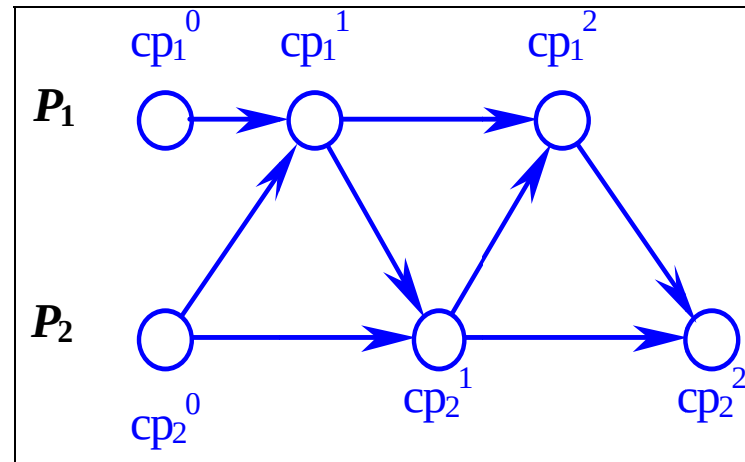


- we use checkpoint graph here but not extended checkpoint graph
- why not?



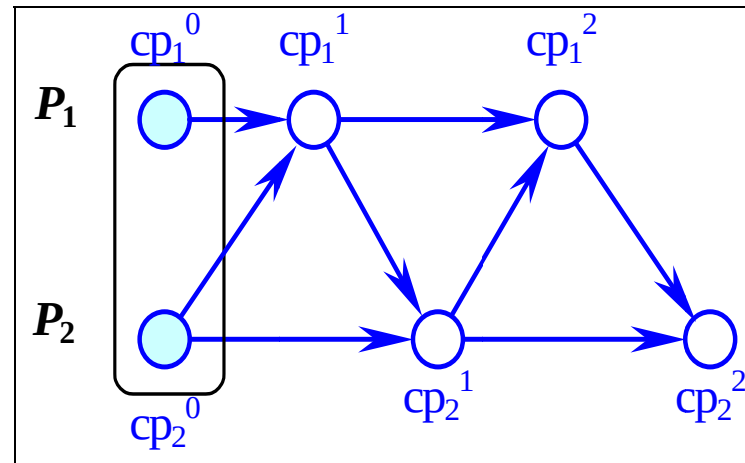
Non-obsolete checkpoints

- are all non-obsolete checkpoints possibly useful for future recovery?
- what about this?





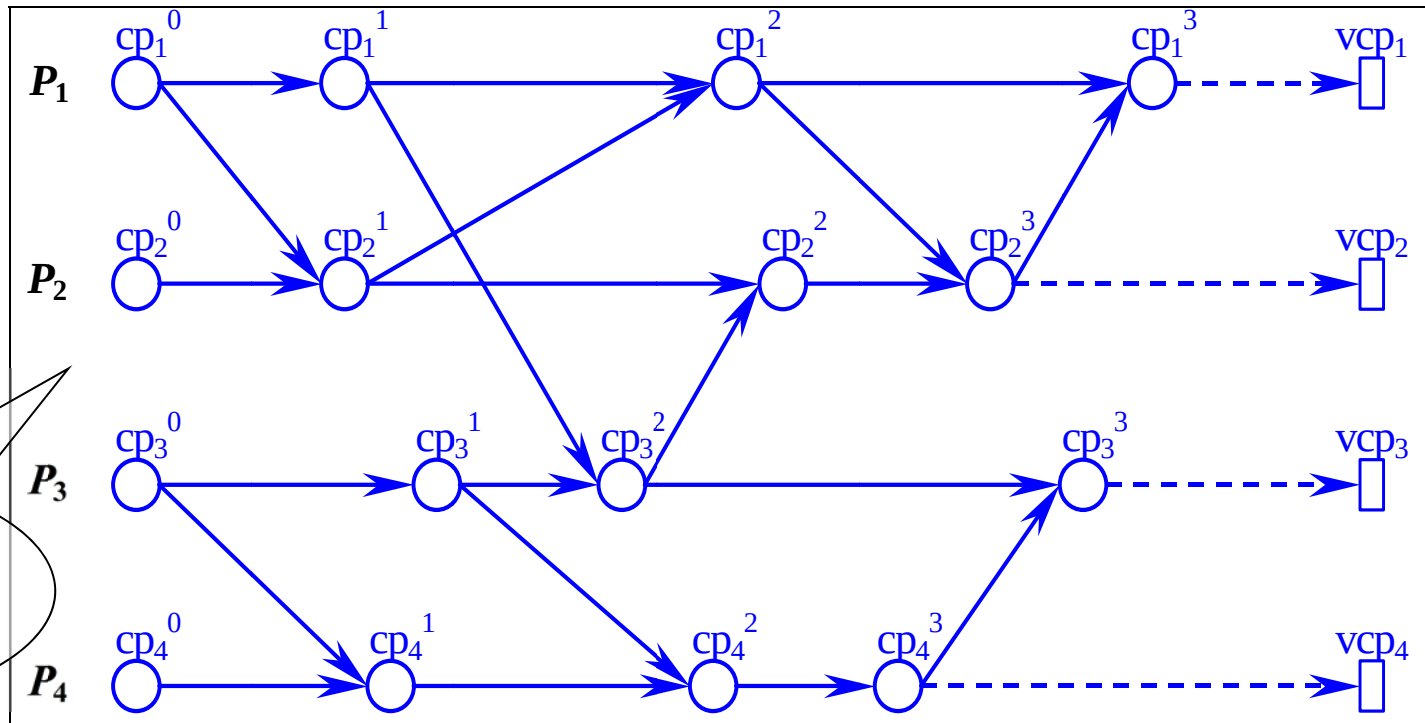
- total domino effect – only initial checkpoints constitute a RL



- z-dependency from cp_1^2 to cp_2^2 and the one from cp_2^1 to cp_1^2 imply that cp_1^2 is inconsistent with any checkpoint of P_2 , thus it will never belong to any recovery line and therefore is discardable
- similarly, cp_1^1 and cp_2^1 are discardable
- in case of domino effect situation, a large number of non-obsolete checkpoints are kept in stable storage, resulting in large space overhead

**Theorem 3:**

Let N be the number of processes and $\hat{G}$ be the supergraph of a checkpoint graph G . A checkpoint in G is non-discardable iff it belongs to the union of recovery lines of all $\hat{G}$ - vcp_i ($1 \leq i \leq N$).

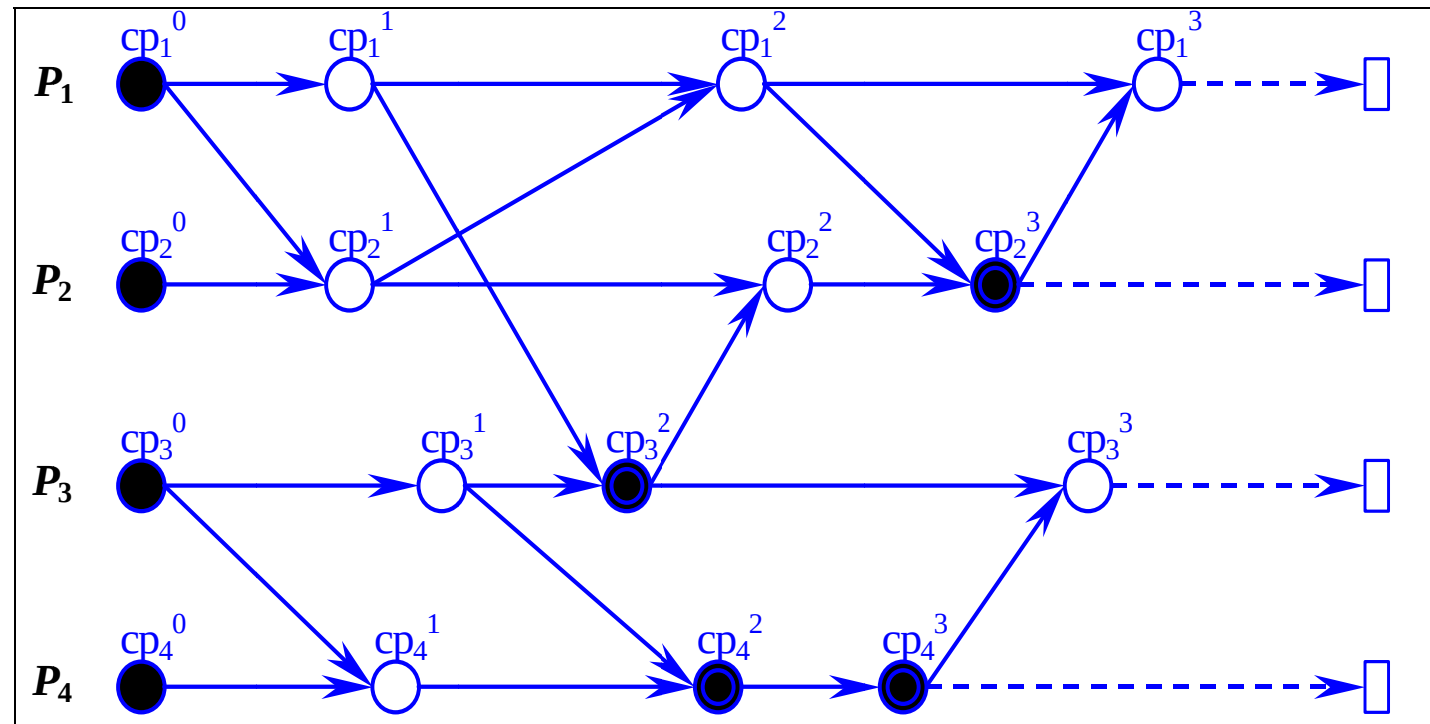


Nadgraf $\hat{G}$ = graf G
sztucznie rozszerzony
o wszystkie wirtualne
punkty kontrolne

Discardable checkpoints:

$cp_1^0, cp_2^0, cp_3^0, cp_4^0$ (obsolete)

$cp_2^3, cp_3^3, cp_4^2, cp_4^3$ (non-obsolete)



BIBLIOGRAPHY

- [1]  T. Juang, S. Venkatesan, “Crash Recovery with Little Overhead”, 11th International Conference on Distributed Computing Systems, 1991, pp. 454-461.
- [2]  Y.-M. Wang, W. K. Fuchs, “Optimistic Message Logging for Independent Checkpointing in Message-Passing Systems”, Symposium on Reliable Distributed Systems – SRDS’92, pp.147-154, 1992.
- [3]  M. Singhal, N.G. Shivaratri, “Advanced Concepts in Operating Systems”, McGraw-Hill, 1994 (2001), ch.12.
- [4]  R. Chow, T. Johnson, “Distributed Operating Systems & Algorithms”, Addison Wesley Longman, 1997, ch.13.